

Практика в ЦБС Петроградского района

Подготовили: Лелло Андрей, Габайдулин Святослав, Махов Святослав
Группа 20б_ПОО/24

Часть 1. Описание репозитория prog-6-ml-service

1.1 Назначение и архитектура

Проект представляет собой веб-сервис машинного обучения (ML-сервис), предназначенный для загрузки обученных ML-моделей и получения предсказаний по входным данным. Сервис реализован в формате REST API и развёртывается с помощью Docker Compose. Архитектура включает три компонента:

- Backend (Python/FastAPI или Flask) — HTTP API для работы с ML-моделями. Предоставляет эндпоинты для загрузки модели, получения предсказаний по JSON и CSV, проверки статуса.
- Frontend — статический веб-интерфейс (HTML/JS), раздаётся через Nginx. Позволяет пользователю взаимодействовать с сервисом через браузер.
- Nginx (reverse proxy) — принимает входящие HTTP-запросы и проксирует их либо на фронтенд, либо на бэкенд в зависимости от пути.

1.2 Структура репозитория

На основе истории коммитов и названий веток восстановлена следующая структура проекта:

```
.
|-- .gitlab-ci.yml
|-- docker-compose-dev.yml
|-- backend/
|   |-- app.py          # основной файл Flask/FastAPI-приложения
|   |-- requirements.txt # зависимости Python
|   |-- Dockerfile
|-- frontend/
|   |-- index.html      # веб-интерфейс
|   |-- nginx.conf      # конфигурация Nginx для фронтенда
|   |-- Dockerfile
|-- README.md
```

1.3 Анализ программного кода бэкенда (app.py)

Отчёт рабочей группы №1 Задание 1

По истории коммитов восстанавливается следующий функционал бэкенда:

GET /status	Проверка работоспособности сервиса (healthcheck). Возвращает статус сервиса и информацию о загруженной модели.
POST /upload-model	Загрузка ML-модели на сервер. Принимает файл модели (например, .pkl, .joblib), сохраняет в память/файловую систему.
POST /predict	Получение предсказания по JSON-данным согласно спецификации. Входные данные валидируются перед передачей в модель.
POST /predict-from-csv	Пакетное предсказание по CSV-файлу. Принимает CSV, возвращает предсказания для всех строк.

Особенности реализации:

- Реализована валидация входных данных
- Добавлена документация в коде и линтер для бэкенда.
- Исправлены замечания линтера в нескольких итерациях, что говорит о наличии проверки стиля кода в CI.

1.4 Конфигурация .gitlab-ci.yml

Файл CI/CD-конфигурации настраивает автоматизированный конвейер с несколькими стадиями. На основе истории коммитов (множественные обновления .gitlab-ci.yml) определяются следующие этапы:

Стадия lint	Запуск линтера для бэкенда (например, flake8 или pylint). Коммиты 'Линт для бэкенда', 'Фикс того, на что ругается линтер' подтверждают наличие этой стадии.
Стадия build	Сборка Docker-образов для backend и frontend. Образы тегируются и публикуются в GitLab Container Registry проекта.
Стадия test	Запуск тестов (предположительно unit-тесты для API-эндпоинтов).
Стадия deploy	Деплой на сервер: пул образов из Container Registry и перезапуск через docker-compose. Используется SSH-подключение к серверу через GitLab CI variables.

Характерные особенности конфигурации:

- Несколько коммитов с сообщением 'Updated .gitlab-ci.yml' свидетельствуют об итеративной отладке пайплайна.
- Коммит 'теперь образы пулятся корректно' указывает на проблемы с тегированием/публикацией Docker-образов, которые были решены.

- Конфигурация поддерживает только ветку master (деплой происходит только из master).

1.5 Конфигурация docker-compose-dev.yml

Файл docker-compose-dev.yml описывает конфигурацию для локальной разработки. Коммит 'Фикс кавычек в docker-compose' подтверждает наличие этого файла. Типичная структура:

Сервис backend	Сборка из ./backend/Dockerfile, проброс порта монтирование volume для горячей перезагрузки.
Сервис frontend	Сборка из ./frontend/Dockerfile, проброс порта 80, использует nginx.conf для раздачи статики и проксирования /api на backend.
Сеть	Общая Docker-сеть для взаимодействия сервисов между собой.

1.6 Конфигурация Nginx (frontend)

Коммит '/predict-from-csv, правки CI и nginx.conf образа frontend' указывает на то, что конфигурация Nginx содержится внутри Docker-образа фронтенда. Nginx выполняет две функции:

- Раздача статических файлов (index.html, JS, CSS) из /usr/share/nginx/html или аналогичной директории.
- Reverse проху: запросы на /api/* проксируются на контейнер backend по внутренней Docker-сети.

